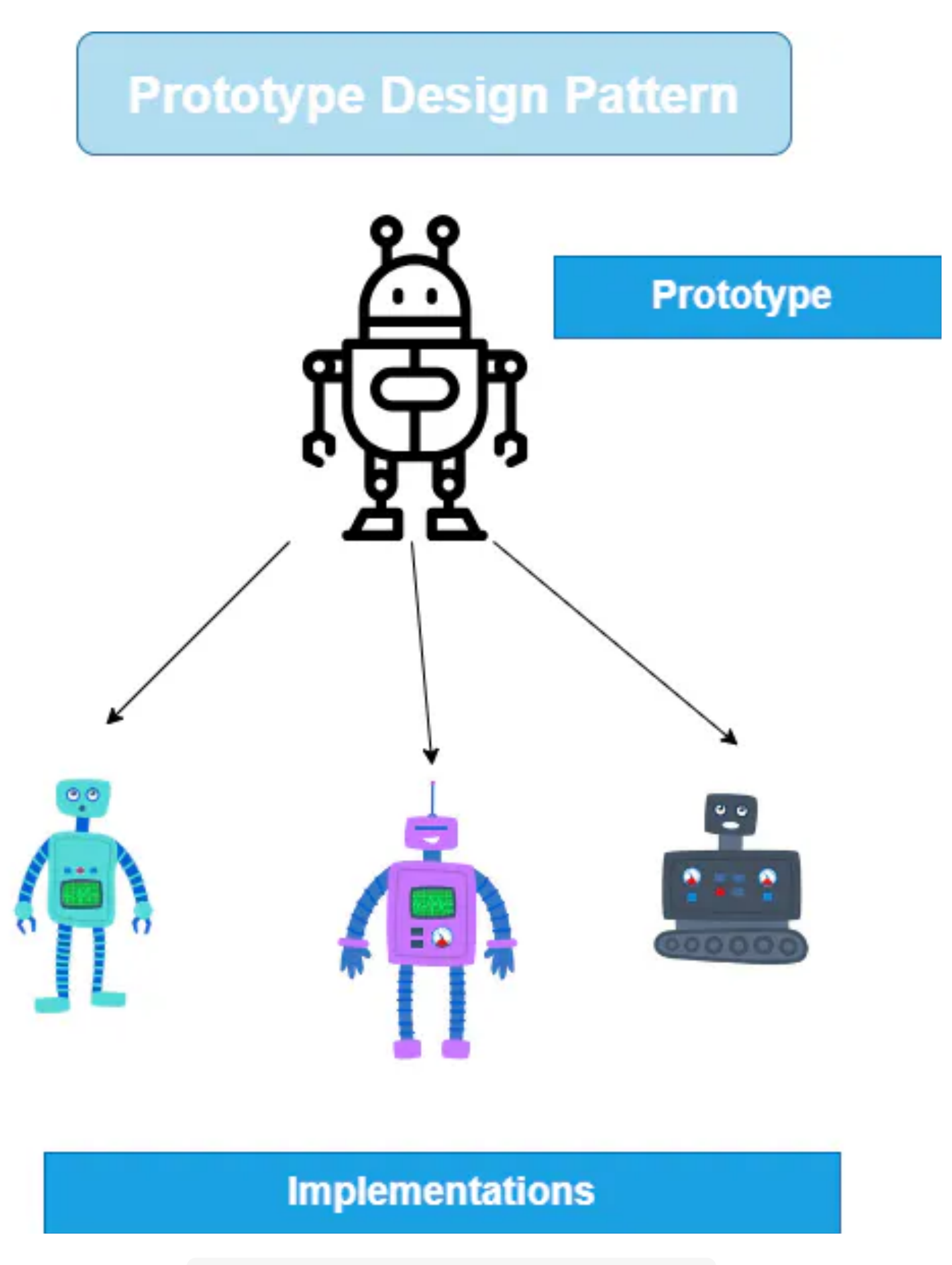
Prototype Pattern

The Prototype Design Pattern is a creational design pattern that involves creating objects based on a template of an existing object through cloning. This pattern is used when creating a new instance of a class is more expensive or complex than copying an existing instance. It's particularly useful in scenarios where objects are dynamically created and the types of objects are not known until runtime.

Real-Life Example



Prototype Pattern - Example

The diagram illustrates the Prototype Design Pattern using the example of robots.

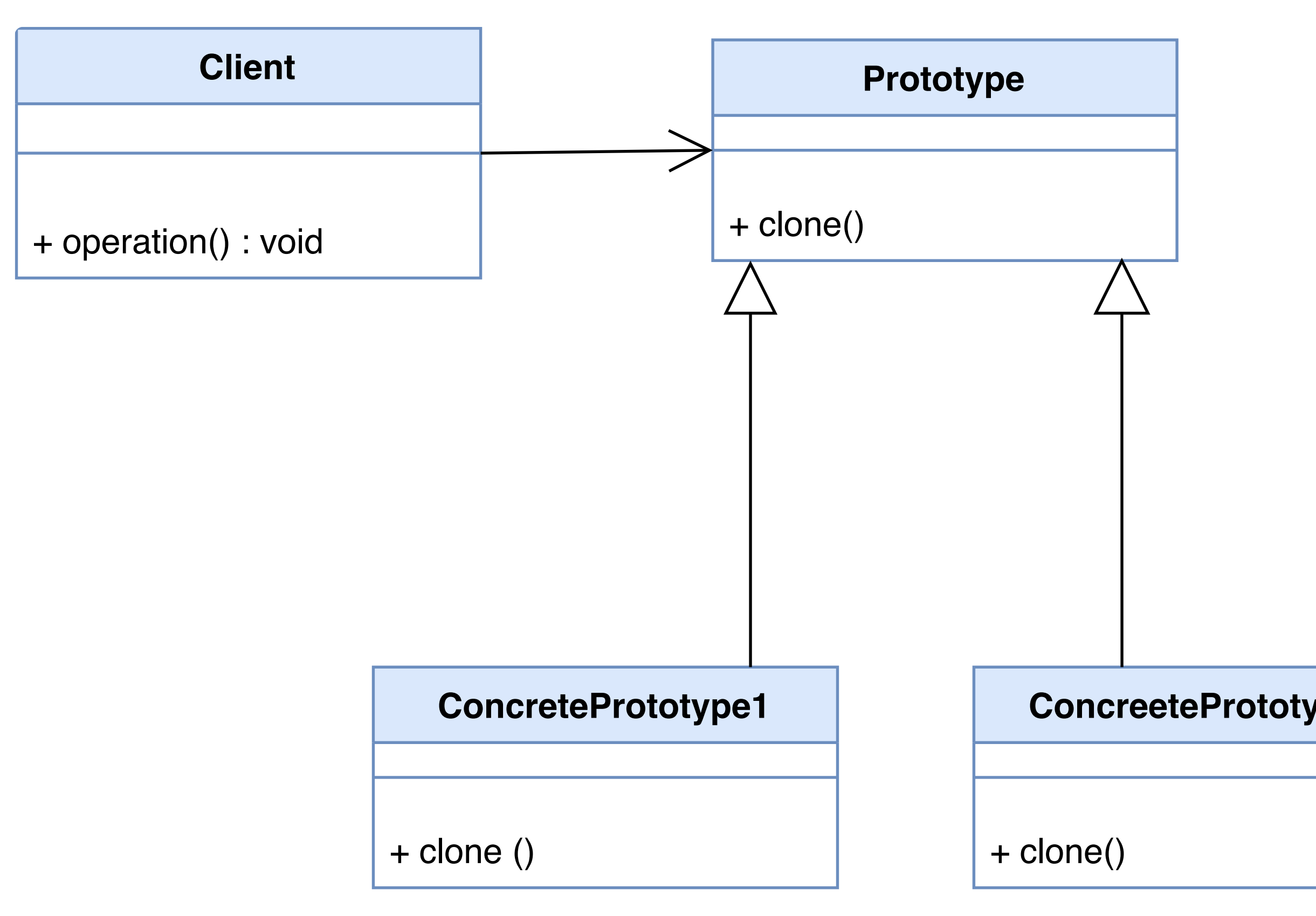
The idea here is that instead of building each of these three robots from scratch, which can be time-consuming and expensive, they are cloned from the prototype robot. This cloning process copies the features and characteristics of the prototype, and potentially, each clone can then be customized further if needed. For example, one might be programmed for talking, another for lifting, and the third for complex calculations.

This is the core concept of the Prototype Pattern: creating new objects by copying an existing object (the prototype) rather than creating new objects from scratch, which can be a more efficient way to produce objects that share similar characteristics or configurations.

Structure of Prototype Pattern

Typically, the prototype pattern includes:

- **Prototype**: An interface with a self-cloning method declared.
- **Concrete Prototype**: An object that implements the Prototype interface and returns a duplicate of itself is referred to as a concrete prototype.
- **Client**: an entity that requests a new object be created by duplicating the prototype.



Prototype Pattern - Class Diagram

How It Works

- The client holds a reference to a prototype object and clones it whenever a duplicate object is needed.
- The cloning process can be either shallow (copying the object's immediate properties only) or deep (copying the object and all objects it references recursively).
- The Prototype pattern allows adding and removing objects at runtime and provides a way to avoid complex inheritance structures or the need to subclass and customize object creation.

Implementation of Prototype Pattern

Let's look at a practical application of the prototype pattern: creating customizable cars. Consider yourself at a vehicle dealership, and you come upon a basic car model. The dealer efficiently clones the basic model and tailors it to your specific preferences, eliminating the need for you to begin from scratch.

Let's look at the pseudocode and implementation of this example in C++, Java, Python, and JavaScript.

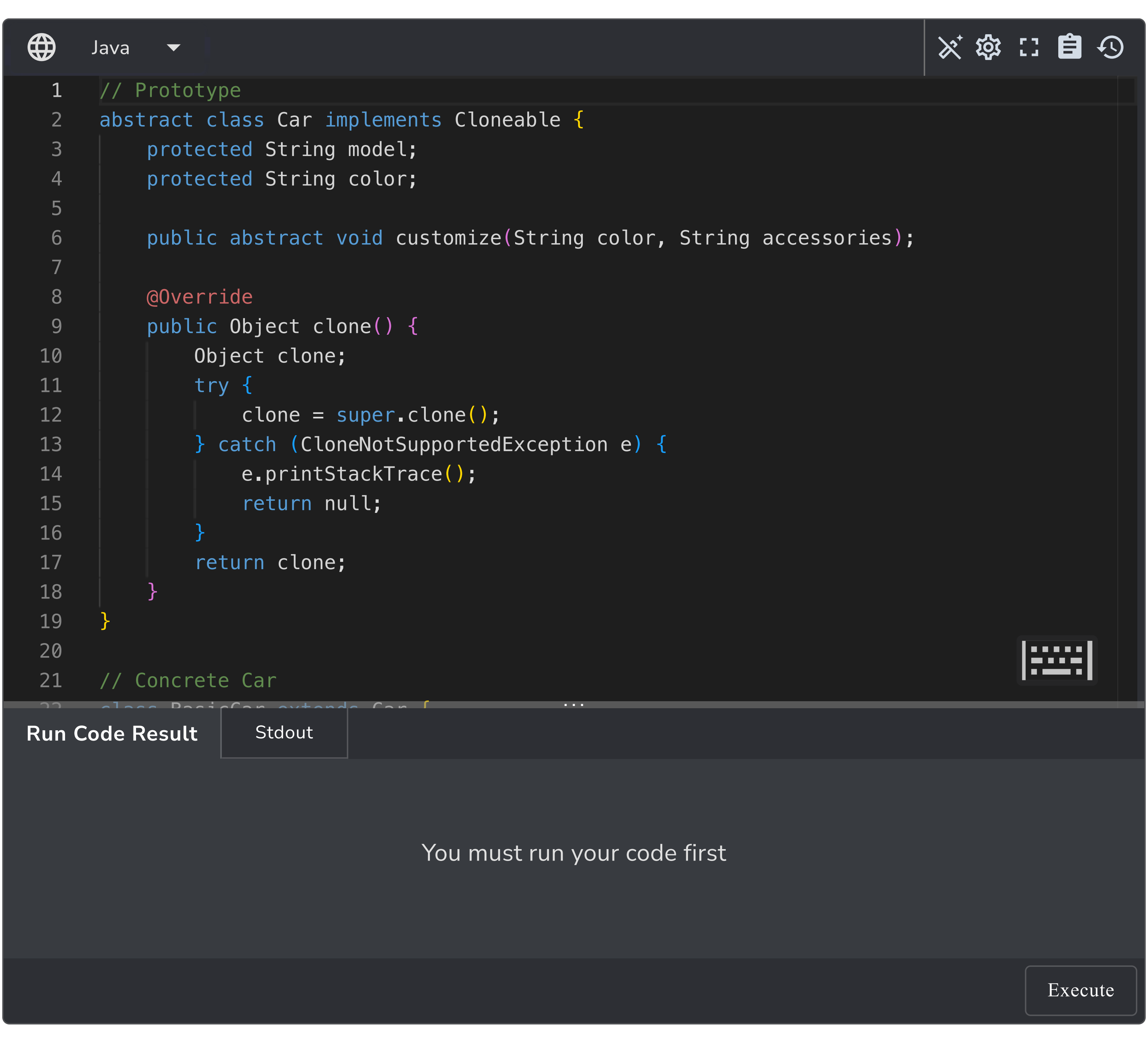
Pseudocode

The pseudocode for the Vehicle dealership example will be as follows:

```
ABSTRACT CLASS Car:

    PROTECTED model, color

    ABSTRACT METHOD customize(color, accessories)

    METHOD clone():
        TRY
            RETURN a copy of this object
        CATCH cloning not supported error
            PRINT error
            RETURN null
        ENDTRY
ENDCLASS

CLASS BasicCar EXTENDS Car:

    CONSTRUCTOR:
        SET model = "Basic"
        SET color = "White"
    ENDCONSTRUCTOR

    OVERRIDE METHOD customize(color, accessories):
        SET this.color = color
        PRINT "Car customized with color:", color, "and accessories:", accessories
    ENDMETHOD
ENDCLASS

MAIN:

    DECLARE basicCar = NEW BasicCar
    DECLARE customerCar = a clone of basicCar

    CALL customerCar.customize("Red", "Sunroof")

ENDMAIN
```

- We have an abstract class `Car` that provides a basis for all cars. This class implements the `Cloneable` interface, enabling duplication of its instances.
- The `customize` method of the `Car` class is an abstract method, which means any concrete subclass of Car must implement it. The properties of the car will be changed using this method.
- The `clone` method is used to duplicate (or clone) a car object. It makes use of the `Object` class's clone method, and if cloning is not supported, it raises an exception and prints a warning.
- `BasicCar` is a concrete subclass of `Car`. In its constructor, it sets default values for model and color. It additionally provides a `customize` method implementation.
- The `main` function is found in the `CarDealership` class. Here, a new object named `BasicCar` is created. Then, `customerCar` is created by cloning this object. The `customize` method is then used to modify the `customerCar`.

Simply said, the code exemplifies the Prototype design pattern, with the original `BasicCar` serving as the prototype and the `customerCar` serving as the cloned instance that may be modified in accordance with the needs of the user.



```java
// Prototype
abstract class Car implements Cloneable {
    protected String model;
    protected String color;

    public abstract void customize(String color, String accessories);

    @Override
    public Object clone() {
        Object clone;
        try {
            clone = super.clone();
        } catch (CloneNotSupportedException e) {
            e.printStackTrace();
            return null;
        }
        return clone;
    }
}

// Concrete Car
```

You must run your code first

Applications of Prototype Pattern

- **Mass Production**: The Prototype Pattern excels when it is necessary to create numerous identical or slightly different instances, much like our artist's sculptures.
- **Resource-intensive objects**: Having a clone prevents the costs associated with objects that need a lot of resources during instantiation.
- **Dynamic Configuration**: When objects must be created at runtime with configurations decided during execution.

Pros and Cons

Pros	Cons
Easy Cloning: Quickly create new objects by copying existing ones.	**Complex Cloning:** Cloning complex objects with interconnected parts can be difficult.
Less Code: Reduces the amount of code needed for object creation.	**Hidden Issues:** Cloning can lead to problems if not all parts of the object are correctly copied.
Flexible Creation: Allows for creating objects dynamically at runtime.	**Management of Clones:** Keeping track of clones and managing their updates can be challenging.
Efficiency: More efficient in cases where object initialization is resource-intensive.	**Shallow vs Deep Copy:** Managing shallow versus deep copies can introduce bugs if not handled carefully.
Prototyping: Facilitates experimentation and prototyping with different object configurations.	**Resource Handling:** Issues can arise if cloned objects hold resources that should not or cannot be duplicated.

Summary

The Prototype Design Pattern is useful for scenarios where object creation is more expensive or complex than duplicating an existing object. It offers a flexible solution to dynamic object creation but requires careful implementation to handle complex cloning scenarios effectively.

← Previous
Builder Pattern
Next →
Summary
☐ Mark as Completed